AWS S3 Automation using Python Boto3

12 May 2026 01:13

AWS S3 Automation Tool using Python Boto3

This project is a cloud automation project developed using Python and AWS Boto3 SDK.

The main goal of this project was to automate repetitive AWS S3 operations such as:

- **creating S3 buckets,**
- **listing buckets,**
- **and uploading files into S3 buckets.**

Normally, these tasks are performed manually from the AWS Management Console, which becomes repetitive and time-consuming in real-world environments. So I wanted to understand how DevOps engineers automate cloud infrastructure using scripting.

In this project, I used Python along with the Boto3 library, which is the official AWS SDK for Python.

First, I created a reusable function to establish connection with AWS services using Boto3 client.

Then I implemented:

- a function to list available S3 buckets,
- a function to create new S3 buckets dynamically,
- and a function to upload local files into the bucket.

The upload functionality uses the upload_file() method provided by Boto3.

I also implemented response validation using HTTP status codes returned by AWS APIs to confirm whether bucket creation was successful or not.

During development, I faced multiple real-world debugging challenges such as:

- variable scope issues,
- incorrect AWS parameter naming,
- function argument mismatch,
- and file path/FileNotFound errors.

I solved these issues by carefully analyzing Python tracebacks and understanding AWS API documentation.

Through this project, I gained practical experience in:

- AWS S3,

- Python scripting,
- Boto3 SDK,
- cloud automation,
- API handling,
- and debugging real-world automation workflows.

This project helped me understand how automation reduces manual effort and improves efficiency in DevOps and cloud environments.

Code Explanation – AWS S3 Automation Tool using Python Boto3

- **Importing Boto3**

import boto3

- boto3 is the official AWS SDK for Python.
- It allows Python programs to interact with AWS services like S3, EC2, Lambda, etc.

- **Creating AWS Service Connection**

```
def get_connection(service):
    return boto3.client(service)
```

Explanation:
- This function creates a client connection for AWS services.
- boto3.client(service) creates a low-level AWS service client.
- Here we pass 's3' to connect with Amazon S3.

Example:
s3_client = get_connection('s3')
This creates S3 client object.

- **Listing Existing Buckets**

```
def show_buckets(s3_client):
    response = s3_client.list_buckets()
```

Explanation:
- list_buckets() is AWS S3 API method.
- It fetches all S3 buckets available in AWS account.
- Response comes in JSON/dictionary format.

Example Response:

```
{
  "Buckets": [
     {"Name": "my-bucket"}
  ]
}
```

◇ Creating S3 Bucket

def create_bucket(s3_client, bucket_name):

Explanation:

- This function creates new S3 bucket dynamically.

```
response = s3_client.create_bucket(
    Bucket=bucket_name,
    CreateBucketConfiguration={
        'LocationConstraint': 'us-west-2'
    }
)
```

Explanation:

- Bucket=bucket_name
  → Defines bucket name.
- CreateBucketConfiguration
  → Specifies AWS region where bucket should be created.
- 'us-west-2'
  → AWS Oregon region.

◇ Checking API Response

if response['ResponseMetadata']['HTTPStatusCode'] == 200:

Explanation:

- AWS APIs return HTTP response metadata.
- Status code 200 means success.

If successful:

print(f"Bucket {bucket_name} created successfully.")

Otherwise:

print(f"Failed to create bucket {bucket_name}.")

◇ Uploading File to S3 Bucket

def upload_to_bucket(s3_client, bucket_name, file_name):

Explanation:

- This function uploads local file into S3 bucket.

s3_client.upload_file(file_name, bucket_name, file_name)

Parameters:

- file_name
  → Local file path.
- bucket_name
  → Target S3 bucket.
- file_name
  → Object name inside S3 bucket.

◇ **Upload Success Message**

print(f"File {file_name} uploaded to bucket {bucket_name} successfully.")

Explanation:

- Displays upload confirmation message.

◇ **Main Execution Flow**

s3_client = get_connection('s3')

- Creates AWS S3 connection.

show_buckets(s3_client)

- Lists existing buckets.

create_bucket(s3_client, 'hemant-ki-new-bucket')

- Creates new S3 bucket.

```
upload_to_bucket(
   s3_client,
   'hemant-ki-new-bucket',
   'file.txt'
)
```

- Uploads local file into S3 bucket.

Final Workflow

Python Script
↓
Boto3 SDK
↓
AWS S3 API
↓
Create Bucket
↓
Upload File
↓
Cloud Storage Automation

◇ **Key Concepts Used**

- Python Functions
- AWS S3
- Boto3 SDK
- API Calls
- HTTP Status Codes
- Cloud Automation
- File Upload Handling
- Debugging & Exception Understanding

◇ **Best Closing Line for Interview**

“This project helped me understand how Python scripts can automate real AWS cloud operations using APIs and Boto3, similar to real DevOps automation workflows.”